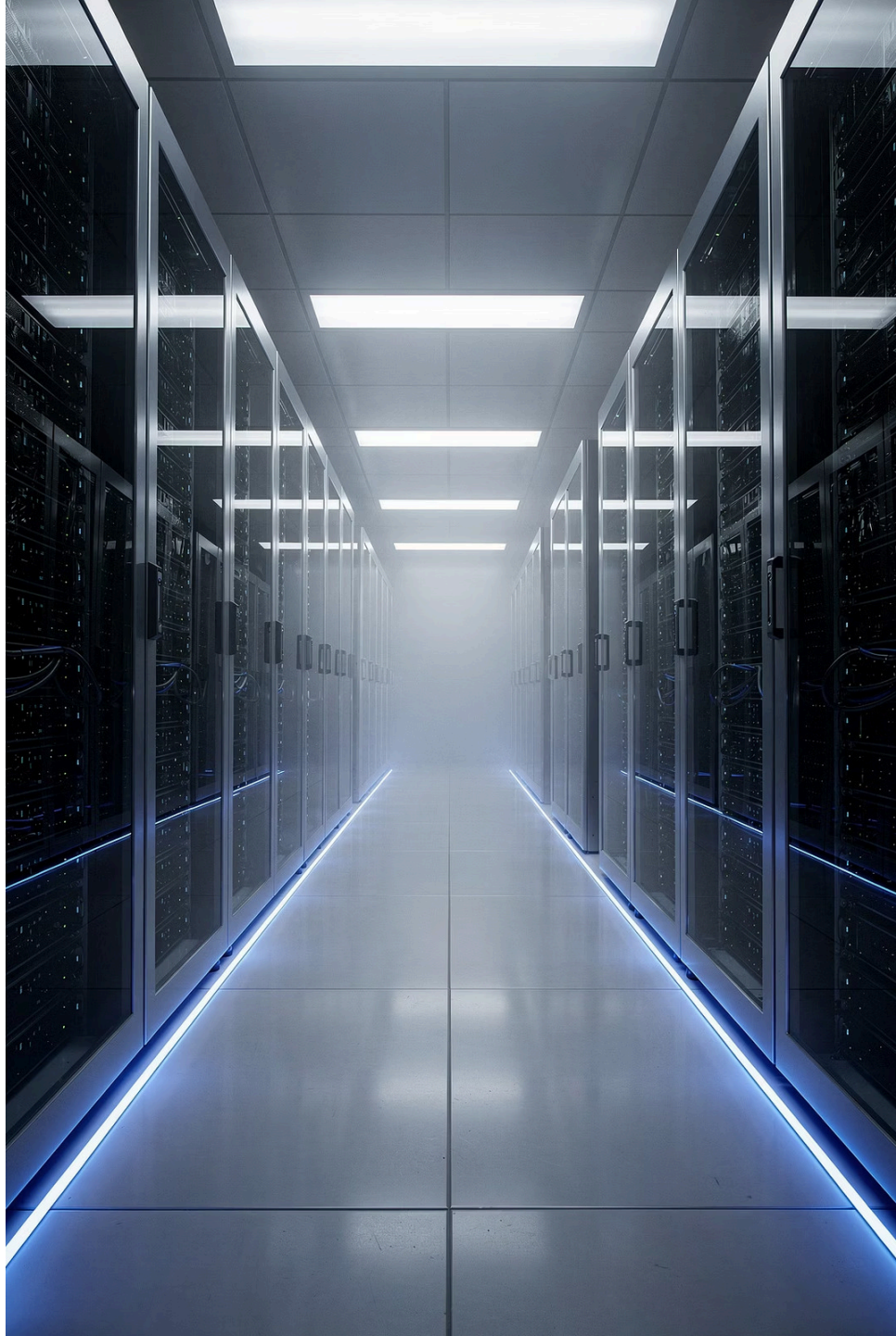
Module 11: Route Guards & Data Resolution

Securing Navigation and Loading Data Efficiently

ANGULAR ENGINEERING | SECURITY | PERFORMANCE



Module Overview

By the end of this module, you will be able to:

1

Understand Route Guards

Learn how Angular intercepts navigation requests.

2

Protect Routes

Restrict access based on authentication and authorization.

3

Use Functional Guards

Implement modern, boilerplate-free guard patterns.

4

Prefetch Data with Resolvers

Load required data before a component renders.

Why Route Guards?

Not every user should access every page. Route Guards control **whether a user can navigate to a route** — backend APIs must still enforce authorization independently.

Protected Areas

- **Admin Dashboard**
- **HR Management**
- **Finance Reports**
- **Employee Portal**

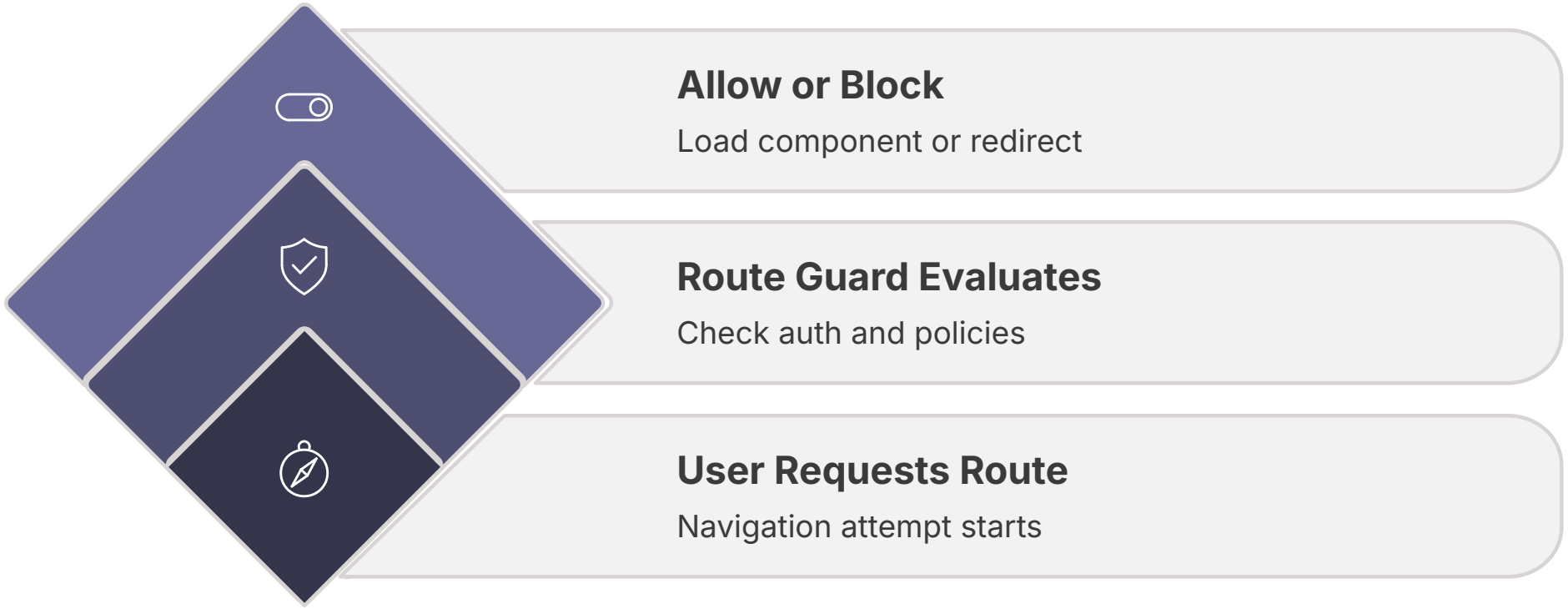
Key Benefits

- Secure application navigation
- Prevent unauthorized access
- Improve user experience
- Enforce business rules consistently



Angular Route Guards

Route Guards allow Angular to evaluate navigation requests and decide whether to allow or block them before a component loads.



Guards run as middleware during the Angular router lifecycle — giving you full control over navigation before any UI is rendered.



CanActivate Guard

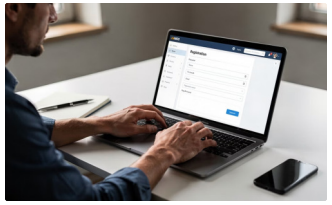
Controls whether a route can be activated. Applied before the component loads, it's the most common guard in Angular apps.

Common Use Cases

- Only authenticated users access the Dashboard
- Only HR users access Employee Management
- Only Admins access Settings and Config
- Subscription or license verification

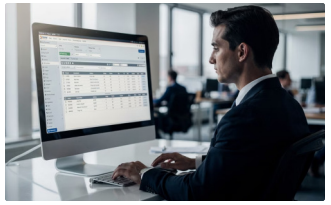
CanDeactivate Guard

Prevents users from accidentally leaving a page with unsaved changes — prompting confirmation before navigating away.



Registration Form

Warn before losing partially entered sign-up data.



Employee Edit

Prevent accidental loss of HR record changes.



Order Entry

Confirm before discarding an in-progress order.



Functional Guards

Modern Angular supports **functional guards** — plain functions instead of classes — reducing boilerplate and improving readability.

Simpler Code

No class scaffolding or interface implementation required.

inject() API

Access services directly inside the guard function.

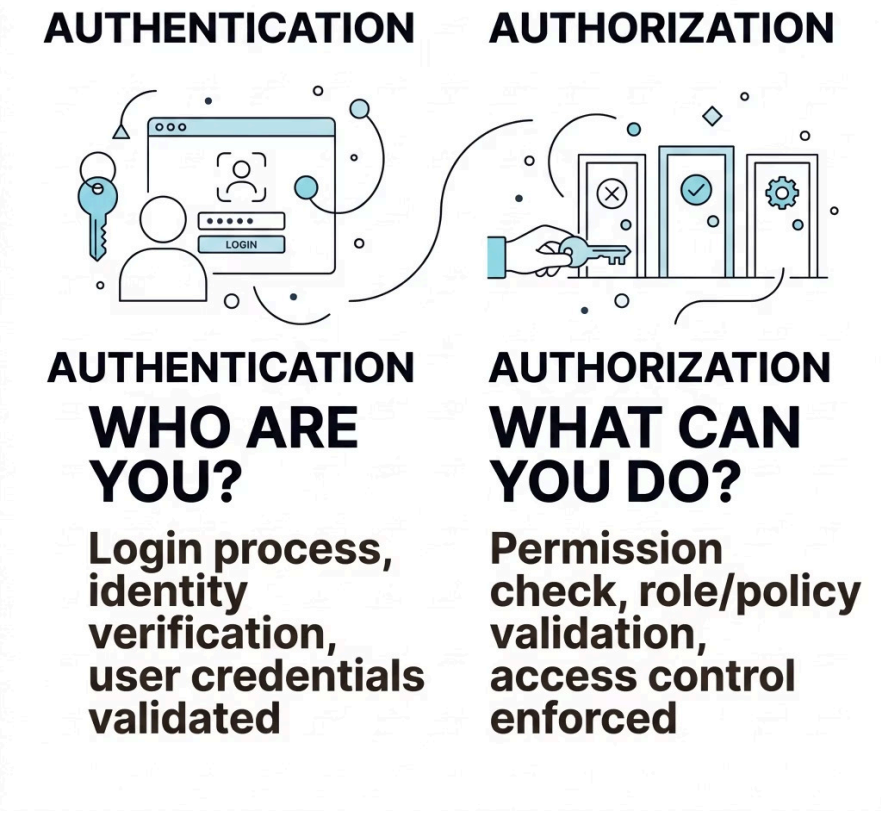
Standalone Ready

Works seamlessly with Angular's standalone component APIs.

Prefer **functional guards** for all new Angular applications unless class-based guards are required by an existing design system.

Authentication vs. Authorization

These two concepts are often confused but serve distinct purposes in securing an application.



Enterprise Example

- **Authentication:** User signs in using Microsoft Entra ID — identity is confirmed.
- **Authorization:** Only managers can approve leave requests — access is scoped by role.

> Authentication identifies the user. Authorization determines what the user can access.

Authentication & Authorization Patterns

Enterprise Angular apps rely on proven identity and access patterns to secure navigation at scale.

Common Enterprise Patterns

- **JWT Authentication**
- **OAuth / OpenID Connect**
- **Microsoft Entra ID**
- **RBAC — Role-Based Access**
- **PBAC — Permission-Based Access**



Route Resolvers

A **Route Resolver** loads required data *before* the component is displayed — so users never see an empty or flickering screen.

Faster Perceived Performance

Components render immediately with all data in place.

Centralized Data Loading

Data fetching lives in the router config, not the component.

Simpler Components

Components receive data ready-to-use, reducing lifecycle complexity.



Data Prefetching Strategies

Choosing the right data-loading strategy depends on how critical the data is to rendering the page correctly.

Strategy	Best For	When to Use
Load After Navigation	Small or optional data	Non-blocking, low-priority content
Route Resolver	Required page data	Data essential to render the view
Lazy Loading	Large feature modules	Modules not needed on initial load
Background Loading	Reference data	Frequently used, cacheable lookups

Use **Route Resolvers** for data that is essential to render the page correctly — avoid showing empty states to the user.

Best Practices

Protect all sensitive routes

Apply guards consistently — never assume a route is safe by default.

Separate authentication from authorization

Keep identity checks and permission checks in distinct, focused guards.

Prefer functional guards

Use the inject() API for clean, testable guard implementations.

Always validate on the server

Client-side guards improve UX — backend validation enforces security.



Module Summary

What We Covered

- **Route Guards**
- **CanActivate & CanDeactivate**
- **Functional Guards**
- **Auth vs Authorization**
- **Enterprise Security Patterns**
- **Route Resolvers & Prefetching**

Up Next — Module 12

HTTP Communication & API Integration

- HttpClient setup and configuration
- CRUD operations with REST APIs
- Error handling strategies
- HTTP Interceptors
- API Service design patterns